Reviewer Pack — Minimal Order of Local Units in Adjoint Group and Communicat…

Entry 28168464ab42 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 13:49:55 UTC

Minimal Order of Local Units in Adjoint Group and Communication Complexity Rank Variance

Entry ID: `28168464ab42` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 13:49:55 UTC

1. Conjecture

Field A (mathematical branch): Adjoint Representation Theory **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> The minimal order of the group of local units in the adjoint representation of a finite group G is linearly related to the variance of the communication complexity rank of its action on an n-bit input space, such that |O(G)| = Θ(Var_R(n)) where Var_R(n) is the variance of the communication complexity rank over all possible actions of G on an n-bit input space.

Rationale (proposer's reasoning):

> Adjoint representation theory provides a way to represent a group action by matrices, which could potentially give rise to insights into communication complexity. The minimal order of local units measures the size of the 'non-trivial' part of this representation and might correlate with the complexity inherent in communicating information.

Taxonomy category: `adjoint_representation` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `62a59f190d8b9a60`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> To support the conjecture, the ratio of the mean minimal order of local units in the adjoint representation to the mean variance of communication complexity rank must be greater than or equal to 0.8 for all n≤40 instances, with no individual instance's ratio exceeding 1.2. To falsify, if any single seed or any combination of seeds results in a ratio less than 0.4 or an individual instance's ratio exceeding 1.2.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "adjoint representation theory" AND "communication complexity rank variance" - "minimal order local units adjoint group" AND "variance communication complexity rank" - "0(Var_R(n))" AND "finite group G"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_group(n):
        # Simple random group generation for demonstration purposes
        elements = list(range(1, n + 1))
        return {i: j for i, j in zip(elements, random.sample(elements, n))}

    def adjoint_representation(group):
        # Adjoint representation as a matrix
        n = len(group)
        adj_rep = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                if group[i][j] == i + 1:
                    adj_rep[i][j] = 1
        return adj_rep

    def communication_complexity_rank(variance):
        # Placeholder function to calculate rank variance
        return variance

    def minimal_order_of_local_units(adj_rep):
        # Minimal order of local units as a simple sum for demonstration
        n = len(adj_rep)
        return sum(sum(row) for row in adj_rep)
```

```
    n = 10
    group = generate_random_group(n)
    adj_rep = adjoint_representation(group)
    variance = communication_complexity_rank(1.0)  # Placeholder value
    min_order = minimal_order_of_local_units(adj_rep)

    return {
        "metric_name": "minimal_order_of_local_units",
        "metric_value": min_order,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41,
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
    else:
        print("RESULT: INCONCLUSIVE reason=mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_293e470a.py", line 65, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_293e470a.py", line 47, in run_trial
    adj_rep = adjoint_representation(group)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_293e470a.py", line 32, in adjoint_repre
    if group[i][j] == i + 1:
       ~~~~~^^^
KeyError: 0
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing data, which prevents us from evaluating the conjecture's support or falsification. | next: Investigate the cause of the crash and rerun the test to gather the necessary data for evaluation.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13930
2	propose	ollama_remote	glm4:latest	0	0	14098
3	propose	ollama_remote	glm4:latest	0	0	12766
4	preregistration	ollama_remote	glm4:latest	0	0	10304
5	novelty	ollama_remote	glm4:latest	0	0	8492
6	novelty	ollama_remote	glm4:latest	0	0	9127
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15652
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11806
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8243
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8304
11	judge	ollama_remote	glm4:latest	0	0	17969

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 130690 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/28168464ab42.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/28168464ab42.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/28168464ab42.tar.gz` (if generated)